Multi-Agent Tool-Use Conversation Generator

Goal

Build an **offline synthetic data generation system** that produces **multi-turn conversations** containing **multi-step / multi-tool tool-use traces**, grounded in tool schemas from ToolBench.

Your system must:

1. **Ingest ToolBench tools/APIs** and construct a **Tool Graph** capturing tools, endpoints, parameters, and semantic groupings.
2. Build a **sampler** on top of this Tool Graph that can propose candidate tool chains that can lead to *realistic* conversations.
3. Implement a **multi-agent generator** that produces conversations of the form:
   - User requests
   - Assistant selects tools/endpoints and emits tool calls across multiple steps
   - Tool outputs can be generated either using LLM or can be taken from actual execution of ToolBench tools.
   - Assistant completes the task
4. Implement an **Agentic Memory** layer that grounds generation in prior tool outputs (session scope) and prior conversations (corpus scope), backed by mem0.

The output is a dataset of conversations suitable for training/evaluating tool-use agents.

Required Deliverables

A repository containing:

- A Python package implementing the pipeline.
- A CLI with commands to:
  - `build` — build registry / graph / index
  - `generate` — generate conversations
  - `validate` — validate conversations

  - `metrics` — compute evaluation metrics on generated data
- Documentation:
  - `README.md` (how to run end-to-end)
  - `DESIGN.md` (architecture + decisions)
- Tests:
  - Unit tests for parsing / validation
  - Unit tests for `MemoryStore`: verify that `add` followed by `search` returns the stored entry, and that entries in one scope are not returned when querying another scope
  - At least one end-to-end test that builds artifacts and generates a dataset of at least 50 samples

---

Required Capabilities

1. Tool Registry

Load ToolBench and normalise tool definitions into a local registry containing, at minimum:

- Tool metadata
- Endpoints / callable units
- Parameter schemas (types, required/optional, enums/constraints when available)
- Response schema or output fields if available

Your loader must handle missing/inconsistent fields (ToolBench variants exist).

---

2. Tool Graph + Tool-Chain Sampler

Construct a Tool Graph with nodes such as:

- Tool
- Endpoint
- Parameter
- ResponseField (when available)
- Concept/Tag

Build different samplers for various types of tool-calling patterns (multi-step, parallel, etc.).

Hard requirement: the generator must use this graph sampler during data generation (not a hardcoded list).

---

3. Offline Tool Execution Model

Because no real tools are executed, implement an offline "execution model" that:

- Validates arguments against the endpoint schema (if available)
- Emits a mock response consistent with the response schema (or a reasonable proxy using LLM)
- Maintains a lightweight session state so later tool calls can reference earlier outputs (IDs, handles, selected items, etc.)

This is required so that multi-step traces are not just random JSON blobs, but actually chain correctly.

---

4. Multi-Agent Conversation Generator

Implement a multi-agent system that generates conversations. It must include at least:

- A **Sampler agent** — proposes tool chains from the Tool Graph
- A **Planner agent** — plans the conversation
- A **User-proxy agent** — behaves as a user
- An **Assistant agent** — produces tool calls or asks clarifying questions
- A **Conversation validator agent** — validates the conversation

Conversation requirements:

- Must include **multi-step** traces (≥ 3 tool calls) and **multi-tool** traces (≥ 2 distinct tools) in a substantial portion of the dataset.
- Must include **multi-turn disambiguation**: the assistant asks clarifying questions before tool calling when the intent is ambiguous or required fields are missing.

---

5. Agentic Memory

5.1 MemoryStore abstraction

Define a `MemoryStore` class (or interface) exposing exactly these two methods:

```
class MemoryStore:
    def add(self, content: str, scope: str, metadata: dict) -> None: ...
    def search(self, query: str, scope: str, top_k: int = 5) -> list[dict]:
        ...
```

The implementation must be **backed by mem0** (`pip install mem0ai`). `scope` is a free-form string used to namespace entries; the two scopes used in this exercise are `"session"` and `"corpus"` (details below). Other components should depend only on the `MemoryStore` interface, not on mem0 directly.

5.2 Session memory (`scope="session"`)

Session memory provides in-conversation grounding: each conversation's tool history is available to the argument-filling step of subsequent tool calls.

Write path — after every tool call completes, write its output to the store:

```
memory.add(
    content=json.dumps(tool_output),
    scope="session",
    metadata={"conversation_id": ..., "step": ..., "endpoint": ...}
)
```

Read path — before constructing the arguments for any tool call that is *not the first step in the conversation*, query the store and inject the retrieved entries into the argument-filling prompt:

```
[Memory context]
{retrieved_entries}

Given the above context and the current tool schema, fill in the arguments
for
{endpoint_name}.
```

Metric — log a `memory_grounding_rate` field in each conversation's metadata, defined as:

> **memory_grounding_rate** = (number of non-first-step tool calls whose argument prompt included at least one retrieved memory entry) / (total number of non-first-step tool calls)

A value of `1.0` means every eligible tool call was grounded in memory; `0.0` means none were. If a conversation contains only one tool call (no non-first-step calls), log this field as `null`.

5.3 Corpus memory (`scope="corpus"`)

Corpus memory provides cross-conversation grounding: the Planner can see what kinds of conversations have already been generated and use that to diversify or specialise future ones.

Write path — after each conversation is fully generated and validated, write a compact summary to the store:

```
# summary_text example:
# "Tools: weather_api, maps_api. Domain: travel planning. Pattern:
        sequential multi-step."
memory.add(
    content=summary_text,
    scope="corpus",
    metadata={"conversation_id": ..., "tools": [...], "pattern_type": ...}
)
```

Read path — before the Planner generates a new conversation plan, query the store and prepend retrieved summaries to the planning prompt:

```
[Prior conversations in corpus]
{retrieved_summaries}

Given the above, plan a new diverse conversation using the following tool
chain:
{proposed_tool_chain}
```

5.4 Diversity experiment

Run the full generation pipeline **twice with the same seed**:

Run	Corpus memory
A	disabled
B	enabled

Compute **at least one** diversity metric across both runs (e.g. pairwise tool-chain Jaccard dissimilarity, n-gram distinct-N on assistant utterances, or entropy over domain/pattern-type labels) and report it in the CLI `metrics` output.

In `DESIGN.md`, include a dedicated section **"Corpus Memory & Diversity Analysis"** that:

- States the diversity metric(s) chosen and justifies the choice.
- Reports the numeric result for Run A and Run B.
- Gives a brief analysis (3–5 sentences) explaining whether and why corpus memory improved (or did not improve) diversity.

Output Format

Generate a dataset (e.g., JSONL) with each record including:

- Conversation messages (role-tagged)
- Tool calls (endpoint identifier + argument dict)
- Tool outputs (mocked but deterministic and chain-consistent)
- Metadata fields:
  - `seed`
  - `tool_ids_used`
  - `num_turns`
  - `num_clarification_questions`
  - `memory_grounding_rate`
  - `corpus_memory_enabled`

How We Will Review

We will run your pipeline end-to-end on our machine with a fixed seed:

Criterion	Weight
Functional correctness	35%
Software engineering practices & code quality	35%
Knowledge graph construction & sampling	10%
Multi-agent system design	10%
Agentic memory implementation & diversity analysis	10%

Appendix: Implementation Notes

- **mem0 setup**: mem0 defaults to an in-process vector store (Qdrant embedded). No external service is required for the exercise. Initialise it as:

  ```
  from mem0 import Memory
  m = Memory()
  ```

  Wrap this inside your `MemoryStore` implementation so the rest of the codebase remains decoupled from the mem0 API.

- **Disabling corpus memory**: expose a flag (e.g., `--no-corpus-memory`) on the `generate` CLI command so that Run A and Run B can be reproduced by reviewers with a single seed argument.

- **Determinism**: mem0's vector search is approximate. For the `memory_grounding_rate` metric, count a retrieval as "present" whenever `search()` returns at least one result, regardless of score threshold.